Software Requirement Specification (SRS)

For a Software Engineering or Software Requirements Engineering lab, I recommend ISO/IEC/IEEE 29148, a key standard for requirements engineering and requirements specification. Explain that an SRS is not simply a description of a software system; it is an agreement about what the system must do and the constraints under which it must operate.

1. Mandatory and essential sections of an SRS

The precise required sections depend on the standard, organizational template, and project. The following is a practical, standards-aligned structure.

1. Introduction

- Purpose of the SRS
- Scope of the software product
- Intended audience
- Definitions, acronyms, and abbreviations
- References
- Document overview, if applicable

2. Overall description and system context

- Product perspective and system boundaries
- Product functions at a high level
- User classes and characteristics
- Operating environment
- Assumptions and dependencies
- Constraints

3. Functional requirements

- All required system structure and functions
- Inputs, processing, outputs, and relevant business rules
- Preconditions and postconditions where applicable
- Error and exception handling
- Authorization and role-specific behavior

4. Non-functional requirements (NFRs)

- Performance and response time
- Security and privacy
- Usability and accessibility
- Reliability and availability
- Scalability and maintainability
- Compatibility and portability
- Applicable legal or regulatory requirements

Specify measurable quality requirements instead of vague words such as “fast,” “secure,” or “user-friendly.”

5. External interface requirements

- User interfaces
- Software interfaces and APIs
- Hardware interfaces, if applicable
- Communication interfaces and protocols
- Specify how the system interacts with users, external software, devices, and networks.

6. Data and business rules

- Data entities and required fields
- Validation rules and data constraints
- Data retention and deletion requirements
- Business policies, calculations, and eligibility rules

7. Verification, traceability, and document control

- Unique requirement IDs
- Acceptance criteria or verification methods
- Traceability to stakeholder needs or sources
- Document version, date, author, and revision history
- Open issues and approval records, where applicable

Important distinction for students: Not every heading above is universally mandatory as a separate section. Some templates combine sections, and some topics are not applicable to every system. However, the requirements themselves must be sufficiently complete for the system being specified.

2. The most important skill: writing correct requirements

Students often write headings correctly but produce poor requirements. Spend substantial teaching time on this part.

Functional requirement example:

Weak requirement

The system should allow students to register for courses easily.

Improved requirement

FR-REG-001: The system shall allow an authenticated student to submit a course registration request for an offered course, provided that the student meets the course prerequisites, has no timetable conflict, and has sufficient credit hours remaining.

The improved version identifies the actor, action, and important conditions. Additional requirements should specify how the system handles failures and confirms successful registration.

Non-functional requirement example:

Weak requirement

The system shall be fast.

Improved requirement

NFR-PER-001: Under a load of 500 concurrent users, 95% of course-search requests shall complete within 2 seconds, measured at the application interface.

The threshold, workload, measurement point, and percentile make this requirement verifiable. The actual targets should be agreed with stakeholders.

3. Characteristics of a good requirement

Use these as a mandatory review checklist.. ISO/IEC/IEEE 29148 provides guidance on well-formed requirements and requirements quality.

Requirements engineering
+1

Characteristic	What students must ensure
Correct	Reflects a genuine stakeholder need.
Unambiguous	Has only one reasonable interpretation.
Complete	Contains enough information to understand the required behavior.
Consistent	Does not conflict with another requirement.
Feasible	Can realistically be implemented within constraints.
Verifiable	Can be checked through testing, inspection, analysis, or demonstration.
Necessary	Contributes to a required capability or constraint.
Traceable	Can be linked to its source and downstream implementation or verification.
Singular	Expresses one principal requirement rather than combining unrelated obligations.

Avoid vague words such as easy, fast, efficient, appropriate, normally, and as soon as possible unless these are defined with objective criteria.

4. Requirement IDs and a requirement specification table

Every requirement should have a unique identifier.

ID	Requirement	Priority	Verification
FR-001	The system shall allow students to search offered courses by course code or title.	High	Test
FR-002	The system shall display available seats for each offered course.	High	Test
NFR-001	95% of search requests shall complete within 2 seconds under the agreed workload.	High	Performance test
SEC-001	The system shall deny access to registration functions for unauthenticated users.	High	Security test

The table can also include the requirement's source, rationale, dependencies, status, and acceptance criteria when useful.